

## Problem Statement

Teams deploying software frequently face operational risk from manual and ad-hoc release procedures. Common issues include missed steps (e.g., tests not executed), unclear accountability, and weak auditability. Existing solutions often require significant setup, assume high DevOps maturity, or do not enforce a human approval gate for automated actions. In academic and small-team settings, these limitations can lead to unstable deployments and delayed releases.

## Proposed Solution

AI DevOps Commander proposes a local-first DevOps command console that translates user intent into a structured deployment plan and enforces plan preview and approval prior to execution. The system is designed around safety policies, auditable logs, and a modular adapter layer to avoid unrestricted shell access.

## Key Features

- Project registry (local path or repo URL) stored in SQLite
- Natural-language command parsing into a plan preview
- Mandatory approval step before enqueueing execution
- Background execution via Redis + RQ with log capture
- Web dashboard to create projects, generate plans, approve and view logs

## Expected Impact

The expected impact is reduced deployment errors through enforced review gates, improved traceability via structured logs, and a clearer operational workflow for non-expert users.

## System Architecture Overview

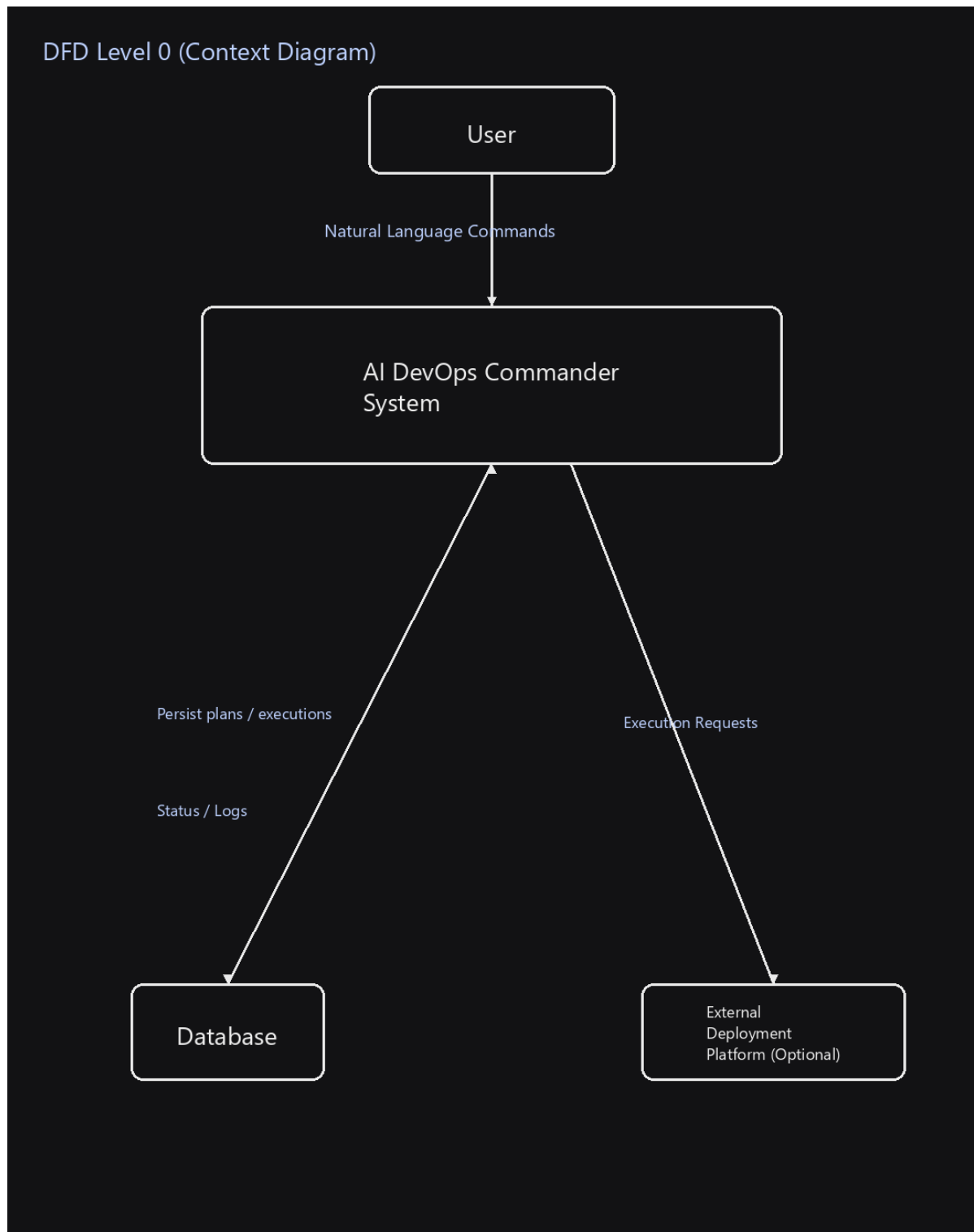
AI DevOps Commander follows a local-first client-server architecture with asynchronous execution. A web dashboard communicates with a backend API, which stores state in a database and uses a background worker to execute approved plans. Deployment can optionally target external platforms when configured.

## Main Components

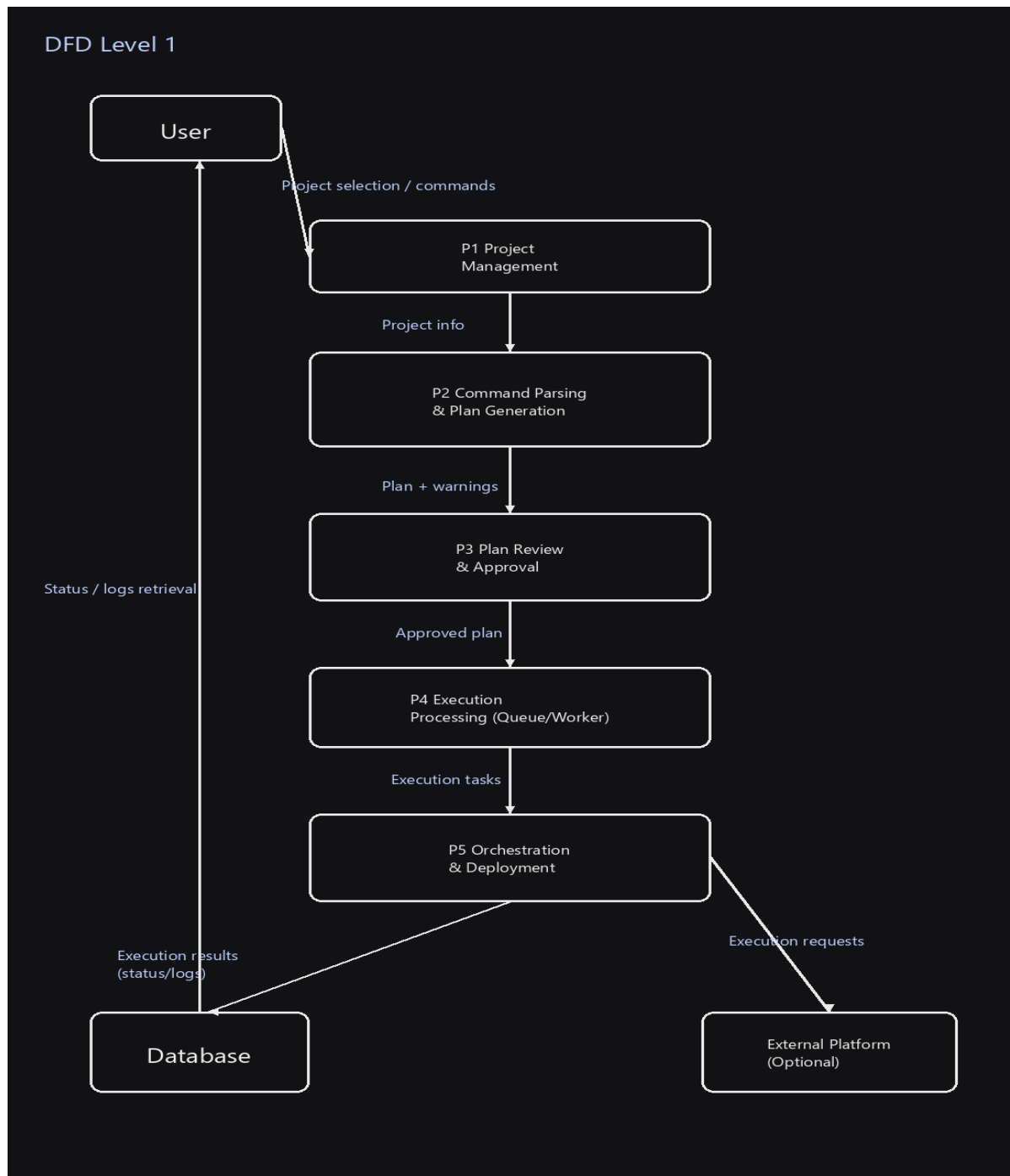
- Web Dashboard (UI): command input, plan preview, approvals, logs view
- Backend API: plan creation, approvals, execution status/log retrieval
- Database: projects, plans, executions, logs
- Queue + Worker: asynchronous plan execution
- Orchestrator / Deployer: dry-run by default; optional real deployments

## Data Flow Diagram (DFD)

### DFD Level 0 (Context Diagram)



## DFD Level 1



## Tools and Technologies Used

| Category                | Tool / Technology       | Purpose                                                                           |
|-------------------------|-------------------------|-----------------------------------------------------------------------------------|
| Programming Languages   | Python                  | Backend API, queue worker, orchestration logic                                    |
| Programming Languages   | TypeScript              | Frontend UI and shared typings                                                    |
| Backend Framework       | FastAPI                 | REST API endpoints                                                                |
| Database / ORM          | SQLite + SQLAlchemy     | Stores projects, plans, executions (local)                                        |
| Queue / Background Jobs | Redis + RQ              | Executes approved plans asynchronously                                            |
| Frontend Framework      | Next.js + React         | Web dashboard for command entry, plan preview, logs                               |
| Deployment Providers    | Vercel API / Render API | Triggers cloud deployments when configured                                        |
| LLM                     | OpenAI / Gemini         | Provider selection exists in settings; command parsing is currently deterministic |
| Container               | Docker Desktop          | For running Redis locally (per docs)                                              |

## Current Implementation / Progress

### Features Completed

- Backend health endpoint: GET /health
- Project registry: POST /projects and GET /projects
- Command parsing: POST /commands/parse (deterministic MVP)
- Plan preview warnings (MVP policy checks)
- Plan approval flow: POST /executions/approve/{plan\_id} enqueues background job
- Execution status retrieval: GET /executions/{execution\_id}
- SQLite persistence (projects, plans, executions)

- RQ worker implementation compatible with Windows
- Frontend dashboard: projects UI, command console, plan preview, live log view

### Features In Progress

- Execution history listing in UI (History panel is mocked; backend listing endpoint not present)
- Richer orchestration steps beyond dry-run logging
- Cloud deployment configuration hardening and end-to-end verification

### Features Pending / Planned

- LLM-backed structured parsing (provider configuration exists; parser is currently rule-based)
- SOP/RAG validation using embeddings and a vector database
- Monitoring integration (Prometheus/Grafana documented; not implemented in current code)
- Automated rollback logic
- Authentication/authorization
- Containerization assets and deployment automation (infra/docker is a placeholder)

### Code Modules Implemented (Repository Mapping)

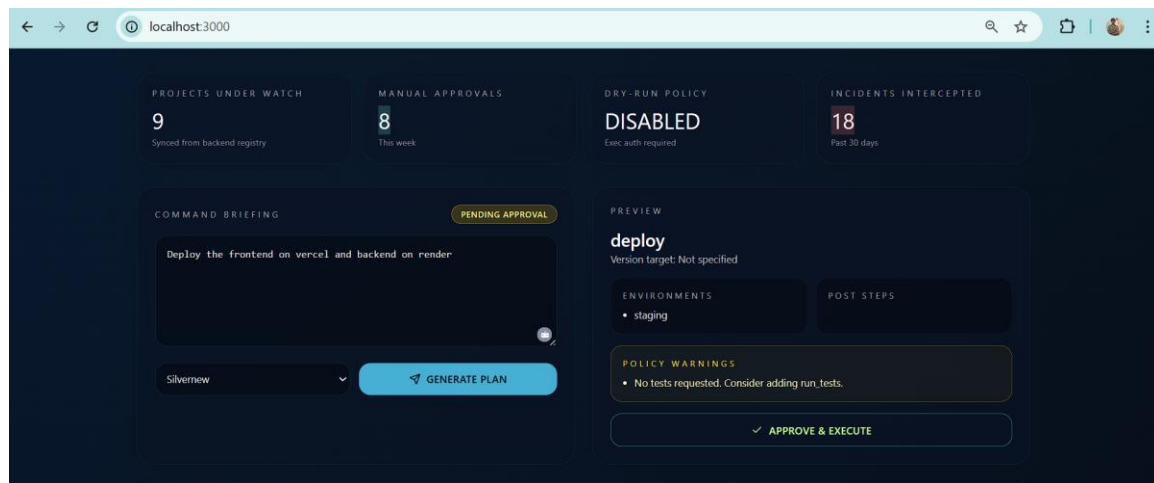
Backend (apps/backend/app):

- api/: routers for projects, commands, executions
- persistence/: SQLAlchemy tables + repository functions
- queue/: Redis connection, RQ queue and worker, task endpoint
- services/: command\_interpreter (MVP), rag\_advisor (stub), orchestrator, deployers
- common/: settings loader and structured logging

Frontend (apps/frontend):

- app/: main dashboard page
- components/: command console, projects panel, stat grid, live log view
- lib/: API client helpers, config, DTO types

## Latest Outputs



### Current Working Features (Observable Outputs)

- Health check output: GET /health returns a JSON status payload.
- Plan generation output: POST /commands/parse returns a plan preview with action, environments, post steps, warnings, and status=pending\_approval.
- Execution output: approving a plan creates an execution and produces execution logs (including DRY\_RUN and provider details).

### Future Scope

Planned enhancements based on repository documentation and current gaps include:

- Replace deterministic command parsing with an LLM-backed structured extractor behind the same API
- Implement SOP/playbook ingestion and vector search to generate policy warnings and enforcement
- Add monitoring-based watchdog and policy-driven rollback workflows
- Expand orchestration steps and add execution history endpoints
- Improve scalability by migrating SQLite to Postgres and formalizing migrations

### Conclusion

At the time of this first review, AI DevOps Commander has a functional MVP demonstrating the intended human-in-the-loop workflow: project onboarding, natural-language plan generation, approval gating, and background execution with log visibility via a web dashboard. Execution safety is emphasized through dry-run defaults and explicit enablement requirements for local execution. The project is ready for the next milestone: integrating an LLM-backed parser, implementing SOP/RAG validation, adding monitoring and rollback, and completing end-to-end deployment history and security controls.